

Preentrega: Multímetro de Interfaz Directa con Calibración y Captura de Datos

Nombre Apellido; Nombre Apellido
Sistemas Embebidos
Universidad Nacional de La Plata



Resumen—Este documento presenta el estado de avance del Trabajo Especial 2 (TE2) al momento de la preentrega obligatoria. Se describe el esquema de tareas implementado sobre FreeRTOS (CMSIS-RTOS2), los tamaños de stack y heap asignados preliminarmente, la tarea de diagnóstico completamente funcional y el mecanismo utilizado para medir el factor de utilización de CPU mediante un analizador lógico. Se incluyen además secciones con encabezados reservados para elementos que se incorporarán en etapas posteriores del trabajo (esquemático, FSM de GUI final, capturas del analizador lógico y de la pantalla).

I. INTRODUCCIÓN

I-A. Consigna general del TE2

El TE2 exige implementar un multímetro con **autorrango** para resistencias y capacidades, con tres modos de operación (configuración, medida y diagnóstico), una pantalla OLED SSD1306 dividida en dos secciones (valor medido / gráfica en tiempo real), y control de usuario mediante encoder y pulsador con una respuesta menor a 20 ms. La medida de los componentes se basa en ciclos de **carga y descarga RC**, siguiendo el esquema de calibración descrito en las Figs. 13 y 14 del pdf proporcionado por la cátedra, que involucran varios ciclos de carga y descarga sobre resistencias/capacitores de referencia conocidos).

A diferencia del Trabajo Especial 1 (TE1, utilizado únicamente como referencia de estructura), el TE2 exige explícitamente el uso de un sistema operativo de tiempo real (RTOS), con al menos cuatro tareas y un elemento de sincronización entre tareas o entre tarea e interrupción.

I-B. Alcance de esta preentrega

La cátedra solicita, para la preentrega obligatoria:

1. El esquema de tareas planteado, funcionando con las tareas propuestas implementadas en el RTOS.
2. La explicación de los tamaños de stack preliminares asignados a las tareas y del heap del sistema operativo.
3. La tarea de diagnóstico completamente implementada, con muestra de datos en pantalla funcionando.
4. La demostración de la medida del factor de utilización con el analizador lógico.

Los cuatro puntos se encuentran implementados y funcionando sobre el microcontrolador STM32 con FreeRTOS (CMSIS-RTOS2). En las secciones siguientes se detalla cada uno.

II. ARQUITECTURA GENERAL DEL SISTEMA

El sistema separa su lógica en **cuatro tareas** de FreeRTOS sincronizadas y comunicadas por **colas de mensajes** y un **semáforo**. Una única variable de estado de **FSM de navegación** (FSMState) gobierna qué se muestra en pantalla, y es actualizada por la tarea de interfaz de usuario a partir de eventos de encoder y pulsador. La adquisición de resistencia corre en una tarea de tiempo real independiente (Task_PROCESSING), que usa al TIMER y ADC.

II-A. Esquema de tareas propuesto

Las cuatro tareas mínimas exigidas por la consigna (interfaz de usuario, graficación/salida, procesamiento y diagnóstico), junto con las colas de mensajes y el semáforo que las conectan.

`task_DISPLAY` centraliza todo el renderizado sobre la OLED: prioriza la cola `alerta` (si hay una alerta pendiente la muestra por encima de cualquier otra pantalla) y luego atiende `myQueue01` o `process2display` según el estado de la FSM de navegación.

MyQueue01 es la encargada de indicarle al display en qué página del menú se encuentra el sistema. La `task_DISPLAY` imprime en pantalla en función del estado y la página obtenidos.

process2display será la comunicación entre la tarea `processing`, que obtiene los datos de resistencia, y el `task_DISPLAY`, enviando un `uint16` que corresponde a la medición actual. La pantalla mostrará este resultado en una mitad de la pantalla y hará un gráfico en la segunda mitad.

diag2display es la comunicación entre `texttt-task_DIAGNOSTICO` y `task_DISPLAY`. Cuando el sistema está en el estado diagnóstico se imprimen en pantalla los tamaños de las stacks en páginas distintas a las que el usuario puede acceder rotando el encoder.

alerta es una cola que se ocupa de los mensajes de error que envía diagnóstico cuando se cumple que el tamaño de alguna stack o de la heap es muy chico o el factor de utilización es muy alto. Es la de más alta prioridad dentro de `task_DISPLAY` y evita que se imprima cualquier otro contenido siempre que haya un mensaje en la cola de alerta.

La Tabla I resume el rol de cada cola.

Cuadro I
COLAS DE MENSAJES IMPLEMENTADAS.

Cola	Payload	Productor → Consumidor
myQueue01	screenMsg_t	Task_GUI → task_DISPLAY
process2display	uint16_t	Task_PROCESSING → task_DISPLAY
diag2display	diagnosticMsg_t	Task_DIAGNOSTIC → task_DISPLAY
alerta	alertaMsg_t	Task_DIAGNOSTIC → task_DISPLAY

II-B. Prioridades asignadas

La Tabla II resume las prioridades asignadas a cada tarea y su justificación. Se buscó que la tarea con requisitos de temporizado más estrictos (adquisición de la señal RC) tenga la prioridad más alta, y que el renderizado de la pantalla, menos crítico en tiempo, tenga la más baja.

Cuadro II
PRIORIDADES ASIGNADAS A CADA TAREA.

Tarea	Prioridad	Justificación
Task_PROCESSING	AboveNormal	Temporizado crítico: adquisición ADC/TIM durante la carga/descarga del RC.
Task_DIAGNOSTIC	Normal	Debe correr periódicamente (50 ms) sin ser bloqueada por la GUI.
Task_GUI	BelowNormal	Responsividad del encoder (≤ 20 ms), pero menos crítica que la adquisición.
task_DISPLAY	Low	Display OLED por I2C. Tolera mayor latencia sin afectar la medición.

II-C. Elemento de sincronización: semáforo contador

Se utiliza un semáforo contador, `myCountingSem01` (capacidad máxima 2, valor inicial 0), como elemento de sincronización entre `Task_GUI` y `Task_PROCESSING`. Al **entrar** al estado `STATE_RUN` de la FSM de navegación, `Task_GUI` libera al semáforo, habilitando la medición. Al **salir** de `STATE_RUN`, lo consume sin bloquear, deteniéndola. Mientras no hay medición activa (`param_a_medir_global == NONE`), `Task_PROCESSING` queda bloqueada en `osSemaphoreAcquire(..., osWaitForever)`, sin consumir CPU.

```
if (estado_actual == STATE_RUN &&
    estado_anterior != STATE_RUN) {
    osSemaphoreRelease(myCountingSem01Handle);
}
if (estado_actual != STATE_RUN &&
    estado_anterior == STATE_RUN) {
    // Consumo inmediato, sin bloquear
    osSemaphoreAcquire(myCountingSem01Handle,
        0);
}
```

Listing 1. Señalización del semáforo al entrar/salir del modo RUN.

II-D. FSM de navegación

Los eventos de encoder y pulsador (`EVENT_H`, `EVENT_AH`, `EVENT_PUSH`, generados en `Leer_Hardware_Encoder()`, con antirrebote por software) son usados de argumentos para la FSM, `fsm_process_event()`, que recibe el estado actual y el evento, y devuelve el próximo estado. Los estados implementados son `IDLE`, `MENU`, `SUBMENU`, `RUN`, `DIAGNOSTIC` y `ERROR`. Esta FSM define, en conjunto con la variable `page`, qué pantalla debe imprimir `task_DISPLAY`.

III. TAMAÑOS DE STACK Y HEAP

III-A. Tamaños de stack

Los tamaños de stack se dimensionaron a partir de un valor base, ajustado luego observando el *Static Stack Analyzer* de STM32CubeIDE y el `HighWaterMark` obtenido en tiempo real. La Tabla III resume los valores actuales.

Cuadro III
TAMAÑOS DE STACK PRELIMINARES ASIGNADOS A CADA TAREA.

Tarea	Stack	Motivo
task_DISPLAY	190×4 = 760 B	Driver SSD1306 + librerías de strings/fuentes + buffer (array) de graficación.
Task_GUI	100×4 = 400 B	FSM de navegación simple + lectura de encoder.
Task_DIAGNOSTIC	100×4 = 400 B	Lectura de <code>TaskStatus_t</code> y cálculo del FU.
Task_PROCESSING	100×4 = 400 B	Máquina de carga/descarga RC, pocas variables locales.

`task_DISPLAY` concentra el mayor consumo de stack por depender de la librería `SSD1306`, de la librería de manejo de strings, y por recibir mensajes de las tres tareas restantes (`Task_GUI`, `Task_DIAGNOSTIC` y `Task_PROCESSING`). Además necesita conservar en un arreglo los últimos valores medidos para graficarlos en tiempo real.

III-B. Heap del sistema operativo

El heap se configura a través del MX y se monitorea su tamaño con `xPortGetFreeHeapSize()` desde `Task_DIAGNOSTIC`. Actualmente el heap se utiliza únicamente para la creación de las colas de mensajes y de los bloques de control de las tareas, todos ellos reservados una única vez dentro de `main()`.

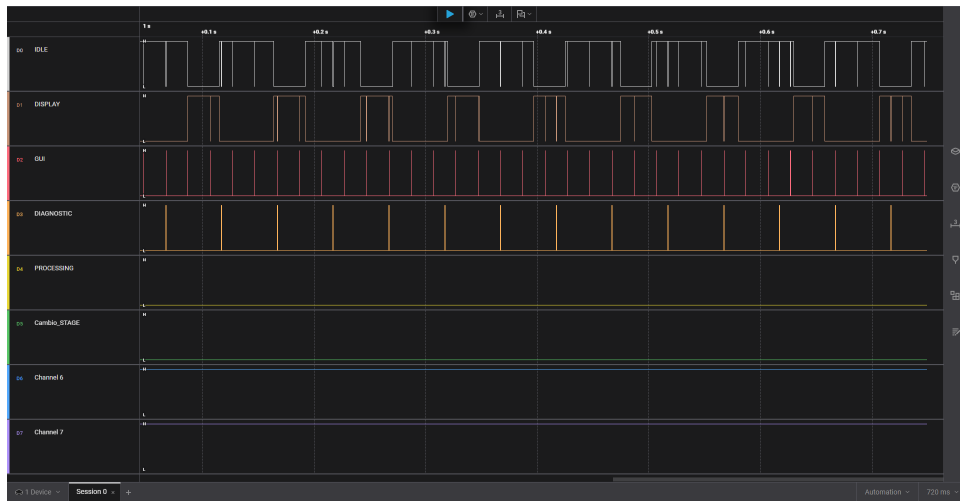


Figura 1. Respuesta Analizador Logico Sin task_PROCESSING

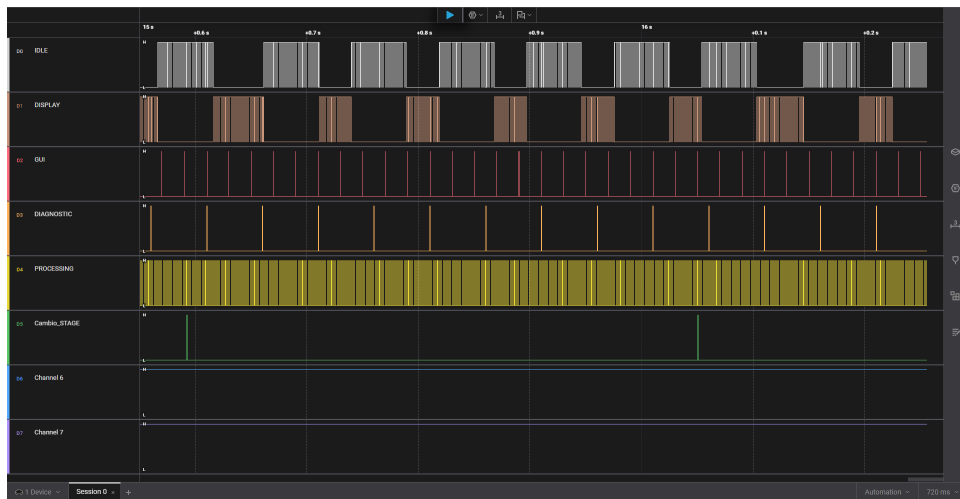


Figura 2. Respuesta Analizador Logico con task_PROCESSING

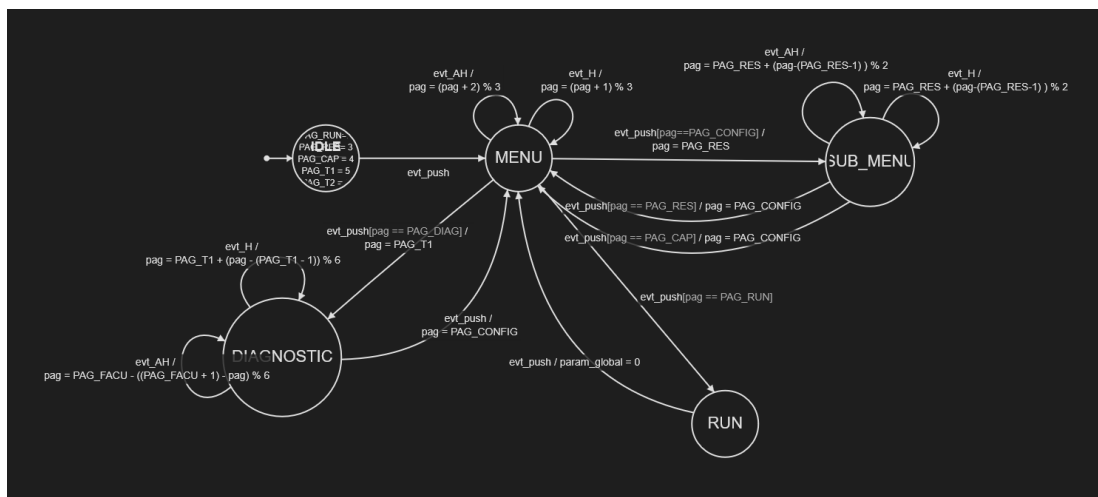


Figura 3. FSM para task_GUI

IV. TAREA DE DIAGNÓSTICO

Cada 50ms (DIAG_PERIODO_MS), la tarea Task_DIAGNOSTIC:

- Lee el *HighWaterMark* de stack de cada tarea con `osThreadGetStackSpace()`.
- Calcula el stack **actualmente libre** de cada tarea con `vTaskGetInfo()`, como diferencia entre el puntero de tope de pila y la base de pila.
- Lee el heap libre del sistema con `xPortGetFreeHeapSize()`.
- Calcula el **factor de utilización (FU)** de la CPU (Sección V).
- Envía los valores en la cola `diag2display` y si corresponde activa una alerta.

```
diagnostic.WaterMark_DISPLAY =
    osThreadGetStackSpace(task_DISPLAYHandle);

vTaskGetInfo(task_DISPLAYHandle, &xTaskDetails
    , pdTRUE, eInvalid);
pxTopOfStack = *(StackType_t**) xTaskDetails
    .xHandle;
pxStartOfStack = xTaskDetails.pxStackBase;

diagnostic.stack_DISPLAY =
    (uint32_t)pxTopOfStack - (uint32_t)
    pxStartOfStack;
```

Listing 2. Cálculo del stack libre de una tarea (patrón repetido para las cuatro).

IV-A. Alertas de stack, heap y FU

La función `devolverERROR()` compara cada magnitud contra un umbral y devuelve el primer código de alerta que corresponda. `STACK_UMBRAL` es el 5 % del stack asignado a cada tarea, se optó por `HEAP_UMBRAL` de 100 bytes y se considero el umbral teórico de FU igual a 69 %

```
uint8_t devolverERROR(diagnosticMsg_t diag) {
    if ((task_DISPLAY_attributes.stack_size
        - diag.stack_DISPLAY) <
        (task_DISPLAY_attributes.stack_size
        * STACK_UMBRAL) / 100) {
        return alerta_StackDISPLAY;
    }
    // ... idem para GUI, DIAGNOSTIC, PROCESSING
    ...
    if (diag.libre_HEAP <= HEAP_UMBRAL)
        return alerta_HEAP;
    if (diag.FU >= 69)
        return alerta_FU;
    return alerta_NONE;
}
```

Listing 3. Generación de alertas por stack, heap y FU.

IV-B. Modo diagnóstico en pantalla

Al ingresar a `STATE_DIAGNOSTIC`, el encoder permite navegar entre páginas: `Task_GUI`, `task_DISPLAY`,

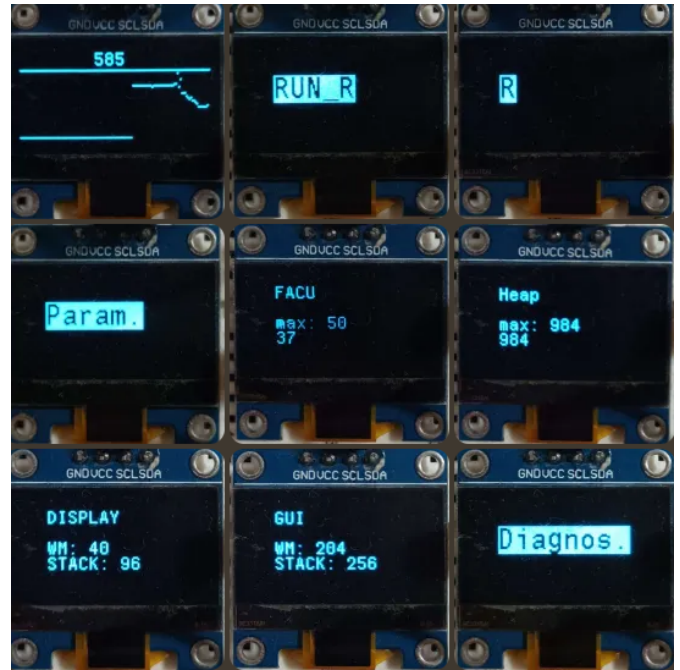


Figura 4. Proyecto en funcionamiento

`Task_DIAGNOSTIC`, `Task_PROCESSING`, `Heap` y `FU`. Cada página muestra su nombre, el *WaterMark* (peor caso histórico) y el stack libre actual. Si llega un mensaje por la cola alerta, se superpone en pantalla independientemente de la página activa.

V. FACTOR DE UTILIZACIÓN Y ANALIZADOR LÓGICO

V-A. Instrumentación mediante hooks de FreeRTOS

Para poder observar la ocupación de CPU con el analizador lógico se hizo uso de los hooks (`traceTASK_SWITCHED_IN/OUT`, vinculados vía `vTaskSetApplicationTaskTag()`), que invocan a `callback_in()` y `callback_out()`. Cada tarea, incluida la tarea `IDLE`, tiene asignado un pin GPIO dedicado que se pone en alto al entrar y en bajo al salir.

V-B. Cálculo del factor de utilización

Haciendo uso del Hook correspondiente a la tarea `IDLE` se utiliza una variable donde se acumulan los ticks de inactividad de la CPU.

El cálculo utiliza una ventana de 50 ms para obtener el FU inmediato y calcula un promedio acumulando el resultado de 30 mediciones.

V-C. Umbral de alerta de FU

El umbral de alerta (69 %) surge de un valor teórico de referencia asociado a la planificación de las tareas periódicas del sistema.

Cuando se accede a la página de factor de utilización dentro de diagnóstico el logic analyzer muestra la siguiente imagen:

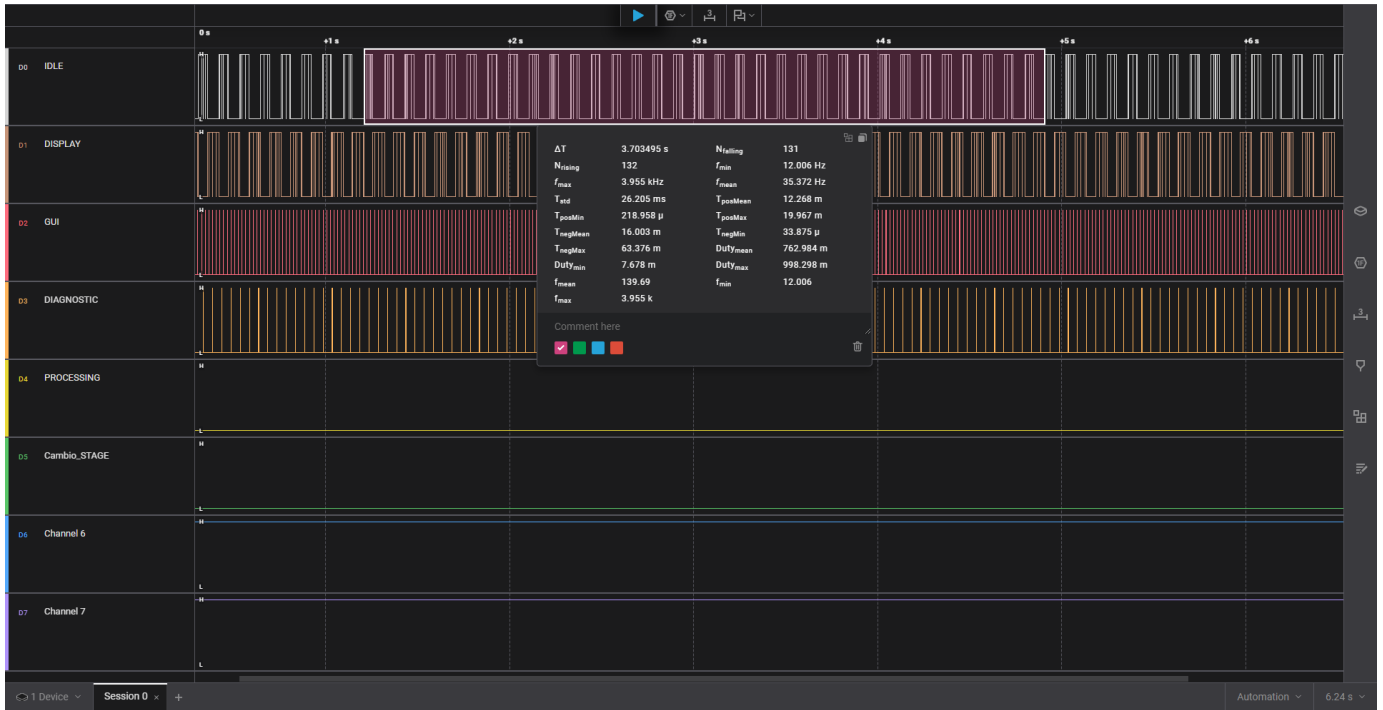


Figura 5. Logic analyzer prueba factor de utilización

Con la extensión “An Extended Pulse Measurements Plugin” by mark omo, se pueden ver algunos datos extras en el Saleae Logic.

Usando el producto entre $T_{negMean}$ y N_{rising} dividido por el ΔT (tiempo total) pudimos obtener una aproximación del tiempo en que IDLE se encontró bajo. Esto sería el factor de utilización.

$$\frac{T_{negMean} * N_{rising}}{\Delta T} = \frac{132 * 16ms}{3700ms} = 0,57$$

En este caso dio 57 %, mientras que el sistema mide 56 % en pantalla.

VI. MÉTODO DE MEDICIÓN R/C

VI-A. Medición de resistencia – estado actual

La medición de resistencia está implementada dentro de Task_PROCESSING como una proceso de cuatro stages, empleando el ADC y el TIM3. Se basa en una calibración de un punto, proceso representado en la fig.6, donde se mide el tiempo de descarga sobre la resistencia bajo prueba (N_x) y sobre una resistencia de referencia conocida R_{c1} (N_c).

$$R_x = \left(\frac{N_x}{N_c} \right) R_{c1}$$

- E1. Carga de $R_i \cdot C$ (umbral 95 % del ADC).
- E2. Descarga a través de R_x , cuenta N_x (umbral 2 %).
- E3. Recarga de $R_i \cdot C$.
- E4. Descarga a través de R_{c1} , cuenta N_c , cálculo de R_x .

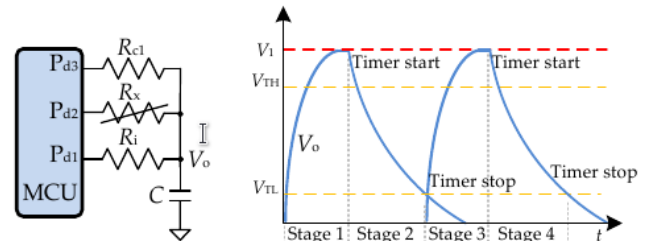


Figura 6. Curvas de carga y descarga

REFERENCIAS

- [1] G. M. Puentes-Conde, E. Sifuentes, J. Molina, F. Enríquez-Aguilera, G. Bravo, G. N. Enríquez, “Direct Interface Circuits for Resistive, Capacitive, and Inductive Sensors: A Review,” *Electronics*, vol. 14, no. 12, p. 2393, 2025. <https://doi.org/10.3390/electronics14122393>